

ANL STAKING PROTOCOL

Security Audit Report — Round #5

Post-audit change verification

Splitting of `initialize` instruction (SBF stack overflow fix)

Date	20 July 2026
Auditor	Grok (xAI) — independent verification
Scope	Working tree after SBF stack-overflow fix (uncommitted)
Package	anl-audit5-repo.zip
Program ID (testnet)	6jiCawqJg5NPR26wCov15tD3HtjKVk1Ao252ZJbZYj1w
Prior rounds	#1–#4 closed for code findings; this is structural delta review

Executive Verdict

The post-audit changes that split the `initialize` instruction into four smaller instructions are **safe for closed testnet**. The intermediate state (GlobalConfig exists, vaults do not yet) is protected by three independent layers. No new critical or high-severity attack vectors were introduced. However, because this is a structural change to a previously audited setup instruction, **formal re-verification is required before immutable mainnet**.

Closed testnet: **ACCEPTABLE** (after integration test update)

Immutable mainnet: **NOT READY** — re-audit of this delta required

1. Context and Motivation

On 20 July 2026, after Audit Round #4 was closed for code findings, the team discovered that `Initialize::try_accounts` overflowed the SBF stack (frame size 6720–7232 bytes against the 4096-byte limit). The program compiled, but the `initialize` instruction was unexecutable on-chain (Access violation in stack frame). The fix splits the single setup instruction into four smaller ones and applies account boxing (`Box<Account>` / `Box<InterfaceAccount>`) to move large account structures from stack to heap.

This audit examines whether the structural change introduces any new security properties or attack surfaces, with particular attention to the newly created intermediate state.

2. Scope of Changes

File	Change	Class
lib.rs	Testnet Program ID + registration of 3 new instructions	Structural
initialize.rs	Split Initialize → Initialize + InitPrincipalVault + InitRewardVault + InitXntVault; boxing	Structural (audited instruction)
stake.rs, lifecycle.rs, fund.rs	Account boxing (<code>Box<Account></code> , <code>Box<InterfaceAccount></code>)	Memory layout
Cargo.toml / Cargo.lock	anchor-lang / anchor-spl 0.29.0 → 0.30.1	Dependencies

3. Findings

A-01 · Intermediate state initialize ↔ init_*_vault — SAFE

Risk hypothesis: After `initialize` the `GlobalConfig` account exists but the three vaults (`principal` / `reward` / `xnt`) do not. Could an attacker exploit this window?

Three protection layers verified in source:

1. **Only authority can create vaults.** All three structures contain `has_one = authority @`

`AnlError::InvalidAuthority:`

- `InitPrincipalVault` — `initialize.rs:144`
- `InitRewardVault` — `initialize.rs:184`
- `InitXntVault` — `initialize.rs:224`

Any other signer is rejected. Front-running vault creation is impossible.

2. **Vaults are PDAs with fixed seeds** (`PRINCIPAL_VAULT_SEED`, `REWARD_VAULT_SEED`, `XNT_VAULT_SEED`) and `token::authority = vault_authority` (itself a PDA). An attacker cannot substitute an arbitrary account — the address is program-derived and the authority is not under attacker control.

3. **Stake requires already-existing vaults.** In `stake.rs` the accounts `principal_vault` and `reward_vault` are typed `Box<InterfaceAccount<TokenAccount>>` **without** the `init` attribute (lines 52, 58). Attempting to stake in the intermediate window fails with `AccountNotInitialized`. There is no code path that writes `principal` into a non-existent vault.

Conclusion: The intermediate window grants the attacker no useful action. The worst case is an operational failure by the authority (setup never completed), leaving the protocol in a state where nobody can stake. This is recoverable by simply finishing the three `init_*_vault` calls. **Not a security vulnerability.**

A-02 · Boxing integrity — CONFIRMED

`Box<T>` only changes allocation (stack → heap) via `Deref/DerefMut`. It does not alter Anchor constraints. All previously present validation rules (`constraint` / `has_one` / `version == ACCOUNT_VERSION` / owner checks on checkpoints) remain intact. Handlers continue to access fields through `auto-deref`; business logic is unchanged.

A-03 · Anchor 0.29 → 0.30.1 migration — NO REGRESSION

The codebase already used the 0.30-style named bump fields (`ctx.bumps.<name>`). Instruction discriminators are computed as `sha256("global:<name>")` and remain stable across Anchor versions for identical instruction names. The three new instructions are correctly registered in `lib.rs:55-64`.

A-04 · Per-network Program ID — CORRECT

Implemented with `cfg` gates in `lib.rs:28-37`:

- `network-mainnet` → placeholder `CvvG4xQq...` (intentionally undeployable)
- `network-testnet` → `6jiCawqJg5NPR26wCov15tD3HtjKVk1Ao252ZJbZYj1w`
- no network feature → dev-only `Fg6PaFpo...`

A-05 · Integration tests inconsistent — MEDIUM (process)

`programs/anl_staking/tests/integration.rs` still constructs the **old** single-step `Initialize` accounts struct that includes the three vaults (approx. lines 177–200). The new `Initialize` structure no longer contains those fields. Consequently the integration suite does not compile / pass against the new code, and the new four-step setup plus the intermediate state have **zero test coverage**.

This is a process finding, not an on-chain vulnerability, but it blocks a green CI and removes the safety net that previously validated the full lifecycle. **Must be fixed before any further push or testnet expansion.**

4. Core Safety Properties — No Regression

The following previously established security properties remain intact after the change:

- XNT epoch-checkpoint model (`end_epoch`, `settlement_cap_index`, `next_funded_epoch` proof)
- Owner check on `UncheckedAccount` checkpoints before deserialization (`lifecycle.rs:71`, `fund.rs:207`)
- `version == ACCOUNT_VERSION` constraints on `PoolConfig` inside `FundXnt`
- Hard compile_error! blocking `network-mainnet` + `test-periods`
- Vault constraints (`mint + authority = vault_authority + token_program`) on all vault-using instructions
- Pause never traps exit paths (`claim / unstake_early / settle_expired`)

5. Recommendations

Priority	Action
MUST	Rewrite <code>tests/integration.rs</code> for the 4-step setup (<code>initialize</code> → <code>3× init_*_vault</code> → <code>create_pool</code>). Without this CI is red / false.
MUST	Add explicit entry in <code>docs/SECURITY-AUDITS.md</code> stating that after Round #4 the structure of <code>initialize</code> was changed (reason: SBF stack overflow) and that this delta requires re-verification before mainnet.
MUST	Honest commit message describing the split as a post-audit structural change, not a “minor fix”.
SHOULD	Synchronise <code>Anchor.toml</code> and documentation Program IDs (testnet addresses are ephemeral — consider keeping them out of the repository).
SHOULD	Consider a dedicated testnet branch so that main remains identical to the last fully audited codebase until re-audit completes.

6. Final Verdict

Phase	Status	Rationale
Closed testnet / limited-value pilot	ACCEPTABLE after integration-test update	Intermediate state safe; no new attack vectors; core safety properties intact
Immutable mainnet	NOT READY	Structural change to a previously audited setup instruction + missing test coverage of the new flow → formal external re-verification required

One-sentence summary: The stack-overflow fix is correct and safe for testnet, but because it changes the structure of a previously audited instruction and introduces a new intermediate state, it does **not** close the audit for immutable mainnet — formal re-verification of this delta is required before production.

This document is an independent security review of a specific post-audit code delta. It does not replace a full re-audit of the protocol before immutable mainnet deployment. All conclusions are based on static analysis of the supplied source snapshot.

Generated 2026-07-20 12:25 UTC by Grok (xAI)